

Empirical Analysis of the Impact of MVP, MVVM and MVI Patterns on Android Code Maintainability through Repository Mining

Daniel Solano Ávila
School of Systems Engineering
Universidad Nacional de Costa Rica
Email: daniel.solano.avila@est.una.ac.cr

Abstract—The Android ecosystem has evolved from monolithic *Activity/Fragment* designs toward separation-of-concerns patterns: Model-View-Presenter (MVP), Model-View-ViewModel (MVVM) and Model-View-Intent (MVI). Despite their growing adoption, quantitative evidence comparing their effect on maintainability is scarce. This paper reports an empirical study that mines open-source Android applications from GitHub and measures, with a single uniform Kotlin/Java static-analysis toolchain, the structural quality of their presentation layer (coupling CBO, cohesion LCOM, complexity WMC) together with a history-based change-proneness signal. Metrics are computed for a curated corpus of code-confirmed MVP, MVVM and MVI applications and compared with non-parametric tests (Kruskal-Wallis, pairwise Mann-Whitney with Holm correction) and Cliff's δ effect sizes. Our extractor is validated against the established CK tool (Spearman $\rho = 0.83$ for CBO and 0.85 for WMC over 1,204 Java classes). Across 28 code-confirmed applications, MVP shows the lowest median presentation-layer coupling (large effect sizes versus MVVM and MVI) while MVVM and MVI are structurally indistinguishable in coupling; MVI attains the simplest individual classes (lowest WMC) at the cost of the lowest cohesion, whereas MVVM concentrates logic in more complex ViewModels. No pattern shows significantly different change-proneness. The study provides objective evidence to guide architectural selection in medium-scale Android development and a fully reproducible mining-and-analysis pipeline.

Index Terms—architectural patterns, Android, MVP, MVVM, MVI, maintainability, code metrics, mining software repositories, change-proneness.

I. INTRODUCTION

Android application development has undergone a significant transformation since the early days of the ecosystem. In its initial stages, the platform encouraged a model where *Activity* and *Fragment* classes concentrated both business logic and visual presentation, giving rise to what are known as *God Objects* or *Massive View Controllers* [1]. While functional for simple applications, this approach presented serious limitations in scalability, testability, and maintenance as projects grew in complexity.

To address these shortcomings, Google and the developer community promoted the adoption of separation-of-concerns patterns. The Model-View-Presenter (MVP) pattern emerged

as one of the first widely adopted alternatives. Later, with the introduction of Android *Architecture Components*, the Model-View-ViewModel (MVVM) pattern gained considerable popularity thanks to its native integration with tools such as *LiveData* and *ViewModel* [2].

With the arrival of *Jetpack Compose*, a declarative UI toolkit, architectures based on unidirectional data flow and immutable state have emerged strongly. The Model-View-Intent (MVI) pattern presents itself as a more predictable solution to the accidental recomposition issues that MVVM can suffer under the declarative paradigm [1].

A. Problem and Justification

Despite the existence of many theoretical debates about the advantages and disadvantages of each pattern, quantitative evidence remains scarce. The lack of consensus and empirical evidence makes it difficult for development teams to choose the right architecture for their projects. A poor choice generates high technical debt, reflected in metrics such as high Coupling Between Objects (CBO) and low cohesion (high LCOM), which complicate long-term maintenance [4], [5].

An objective empirical analysis is needed to compare these architectures based on concrete, measurable metrics. Extracting real data through Mining Software Repositories (MSR) techniques on GitHub allows an objective evaluation of which architecture better contains coupling, preserves cohesion, and limits the change-proneness of the presentation layer, thus going beyond purely opinion-based debates [3].

B. Objective

To empirically analyze the impact of the MVP, MVVM, and MVI architectural patterns on the maintainability of medium-scale Android applications, through the extraction and comparison of presentation-layer static code metrics (WMC, CBO, LCOM) and a history-based change-proneness signal obtained from open-source projects on GitHub.

C. Paper Outline

The remainder of this document is organized as follows: Section II presents the research questions. Section III reviews

related work. Section IV describes the theoretical background. Section V details the repository-mining and static-analysis methodology. Section VI reports the results, and Section VII interprets them and contrasts them with related work. Section VIII discusses threats to validity, and Section IX presents conclusions and future work.

II. RESEARCH QUESTIONS

Based on the identified problem, this study poses the following research questions:

- RQ1:** Which architectural pattern (MVP, MVVM, or MVI) produces the lowest coupling (CBO), the highest cohesion (lowest LCOM) and the lowest complexity (WMC) in the presentation layer of Android applications?
- RQ2:** Are there significant differences in the *change-proneness* of the presentation layer (how frequently its code is modified over the project history) among applications implemented under the MVP, MVVM and MVI patterns?

The first question targets the *internal structural quality* of the presentation layer at a single point in time; the second targets its *evolution* over time, a complementary and directly mineable proxy of maintainability in practice [8]. Runtime performance, considered in earlier proposals, is intentionally left out because it cannot be obtained from repository mining without executing each application; it is revisited as future work (Section IX).

III. RELATED WORK

Vijaywargi and Boddapati [1] performed a qualitative comparison of MVP, MVVM and MVI in modern Android development, concluding that MVI offers greater predictability in state management at the cost of a steeper learning curve. Their analysis, however, did not include quantitative code-quality metrics, a gap this study addresses.

Pradana et al. [2] conducted a comparative performance analysis of MVVM and MVP on a single Android dashboard system, measuring response time and memory usage and reporting statistically significant differences in memory consumption under load. Because the study was limited to one controlled project, its generalization is restricted; we instead mine many real projects, trading runtime measurement for breadth and external validity.

Kalliamvakou et al. [3] documented the promises and perils of mining GitHub, warning about biases such as the prevalence of personal or inactive repositories and the difficulty of inferring causality from observational data. These warnings directly shaped our inclusion/exclusion criteria and manual validation step (Section V).

The object-oriented metrics we adopt originate in the suite of Chidamber and Kemerer [5] (CBO, WMC and the original LCOM), later refined by Henderson-Sellers [6], whose normalized LCOM-HS we use. Filó et al. [4] derived empirical thresholds for these metrics from a large corpus of open-source Java projects; we use those thresholds as comparison baselines.

To compute the metrics consistently across Kotlin and Java we build on the same definitions implemented by the widely used CK tool [7], against which we validate our extractor.

Finally, change-proneness, the frequency with which a class or file is modified, is a well-established external indicator of maintainability: Khomh et al. [8] showed that structurally degraded classes are significantly more change-prone. We mine this signal from version history as our second, evolution-oriented response variable.

IV. BACKGROUND

A. Architectural Patterns in Android

Architectural patterns define how responsibilities are separated among the components of an application. The three patterns central to this study are the following.

MVP (Model-View-Presenter) separates presentation logic into a *Presenter* that mediates between the data model and a passive view, so that the view delegates every logical decision to the presenter.

MVVM (Model-View-ViewModel) introduces a *ViewModel* that exposes observable data streams (e.g., `LiveData` or `StateFlow`); the view subscribes to these streams, which removes the direct view–logic reference. It is the pattern officially recommended by Google for projects built on the Architecture Components.

MVI (Model-View-Intent) implements a strictly unidirectional data flow: the UI state is immutable and is updated only through *intents* that produce a new state, typically via a `reduce` function. It is particularly well suited to Jetpack Compose.

A key observation, confirmed during this study, is that on Android MVI is almost always *built on top of* the MVVM infrastructure (a `ViewModel` holding an immutable `StateFlow`). The patterns therefore form a progression rather than disjoint categories, which has direct consequences for their automated classification (Section V).

B. Code Quality Metrics

To assess structural maintainability we use the following object-oriented metrics [4]–[6].

WMC (Weighted Methods per Class) is the sum of the cyclomatic complexity of all methods of a class; higher values indicate greater complexity and maintenance effort.

CBO (Coupling Between Objects) is the number of other classes a class is coupled to; high CBO hinders unit testing and reuse.

LCOM (Lack of Cohesion in Methods) is the degree to which the methods of a class fail to share instance attributes. We use the normalized Henderson-Sellers variant (LCOM-HS); higher values indicate lower cohesion.

SLOC (Source Lines of Code) and **NOM (Number of Methods)** are size measures used for normalization and characterization.

C. Mining Software Repositories

Mining Software Repositories (MSR) extracts and analyzes data from version control systems and other digital sources to obtain empirical knowledge about software development [3]. In this study GitHub is the primary data source, and rigorous selection criteria are applied to mitigate the biases identified in the literature.

V. METHODOLOGY

The study follows the guidelines for empirical experimentation in software engineering [9]. The whole process is implemented as a reproducible pipeline of Python and Java scripts; all data and scripts are released with the paper.

A. Corpus Selection and Classification

GitHub’s search API is queried for native Android repositories declaring an MVP, MVVM or MVI architecture. Inclusion criteria require a native Android language (Kotlin/Java), an open-source license, at least six months of history and activity within the last twelve months. Repositories are excluded if they are forks, archived, tutorial/sample collections, or *consumable libraries or frameworks* (detected through topics, publishing plugins and a manually curated list); the latter is essential because the most popular “MVP/MVI” repositories are architecture libraries rather than applications. Each candidate is cloned and its pattern is *confirmed from code signals* (presence of `Presenter` classes, `ViewModels` with observable state, or unidirectional-flow markers such as sealed intents and `reduce`), not from its README. Because MVI is implemented on top of MVVM, a precedence rule assigns MVI when its distinctive markers are present. From roughly 7,000 candidate repositories returned by the queries, 232 satisfied the automated eligibility filters; after pattern confirmation and a manual audit removing libraries and misclassified projects, the final corpus comprised **30 applications** (MVVM = 12, MVI = 12, MVP = 6). Genuine MVP applications proved very scarce: every highly starred “MVP” repository was in fact an architecture library, so the MVP group is necessarily small and low in popularity. Table I lists the most popular application of each pattern as an illustration of the corpus and of this popularity gap.

B. Static Metric Extraction

A single tree-sitter-based extractor parses both Kotlin and Java and computes, per class, SLOC, NOM, WMC, CBO and LCOM-HS, using uniform definitions across languages. Only *presentation-layer logic holders* (Presenters, ViewModels, stores/reducers, or classes extending a ViewModel base) with at least one method are aggregated, so that the comparison is equitable across patterns and not diluted by empty marker or pure-data classes. Per-class metrics are aggregated to *per-repository medians* to avoid pseudo-replication (classes within a repository are not independent). Two MVI projects whose state holders use a non-standard naming convention contributed no logic holder and were dropped, leaving 28 applications for analysis (MVVM = 12, MVI = 10, MVP = 6).

TABLE I
REPRESENTATIVE APPLICATIONS IN THE CORPUS (HIGHEST-STARRED PER PATTERN; THE FULL LIST OF 28 IS IN THE RELEASED DATA)

Pattern	Application	Stars	Lang.
MVP	JsonChao/Awesome-WanAndroid	2,719	Java
	RznNike/EyeHealthManager	6	Kotlin
	siempredelao/Distance-From-Me	4	Kotlin
MVVM	TeamAmaze/AmazeFileManager	6,212	Kotlin
	youlookwhat/CloudReader	5,011	Java
	tanujnotes/Olauncher	3,602	Kotlin
MVI	vItzor/TimePlanner	634	Kotlin
	SkyD666/Rays-Android	326	Kotlin
	SkyD666/Raca-Android	117	Kotlin

The disparity in popularity between MVP and the other two groups is itself a finding: genuine, highly starred MVP *applications* are scarce because most popular “MVP” repositories are architecture libraries.

C. Extractor Validation

To establish construct validity, the extractor is run alongside CK [7] on the Java subset of the corpus and the two are compared with Spearman correlation over 1,204 matched Java classes. Agreement is strong for CBO ($\rho = 0.83$) and WMC ($\rho = 0.85$) and moderate for LCOM ($\rho = 0.58$, expected because CK implements a different LCOM variant), confirming that the custom extractor measures the same constructs as an established tool.

D. Change-Proneness

For RQ2, the git history of each presentation-layer file is mined to obtain the number of commits that touched it and the churn (lines added plus deleted), normalized respectively by the repository’s active lifetime and by file size, and aggregated to per-repository medians.

E. Statistical Procedure

Because the samples are independent (different repositories per pattern), the omnibus Kruskal–Wallis test is applied to each metric, followed by pairwise Mann–Whitney U tests with Holm correction and Cliff’s δ effect sizes ($\alpha = 0.05$). This corrects earlier proposals that suggested the Wilcoxon signed-rank test, which assumes paired data.

VI. RESULTS AND ANALYSIS

The analysis covers the 28 applications with measurable presentation-layer logic holders. Table II gives a one-line summary of the headline findings; Table III summarizes the per-repository median metrics by pattern and Table IV reports the statistical tests.

A. RQ1 – Presentation-Layer Structure

Coupling (CBO). MVP exhibits the lowest *median* coupling (6.5), below MVVM (10.5) and MVI (11.0); see Fig. 1. The Kruskal–Wallis test is borderline ($H = 5.36$, $p = 0.069$) and, although Holm-corrected pairwise p -values do not cross $\alpha = 0.05$ given the small samples, the effect sizes are *large* for MVP versus MVVM (Cliff’s $\delta = -0.67$) and MVP versus MVI ($\delta = -0.55$). In contrast, MVVM and

TABLE II
ONE-LINE SUMMARY OF FINDINGS BY RESEARCH QUESTION

RQ	Key finding
RQ1	MVP shows the lowest <i>median</i> coupling (large effect sizes vs. MVVM and MVI, though the omnibus test is borderline). MVVM and MVI are indistinguishable in coupling and differ only in <i>where</i> they place logic: MVVM concentrates it in fewer, more complex ViewModels (high WMC), MVI spreads it across many small but less cohesive classes.
RQ2	No significant difference in change-proneness (commits per month or churn per SLOC) across patterns; presentation-layer evolution appears driven by product and team factors rather than the architectural pattern.

TABLE III
PRESENTATION-LAYER STRUCTURAL METRICS BY PATTERN
(PER-REPOSITORY MEDIANS)

Metric	Pattern	n	Median	Mean	IQR	Min	Max
CBO	MVP	6	6.50	6.67	4.00	3.00	11.00
	MVVM	12	10.50	12.25	6.00	5.50	22.00
	MVI	10	11.00	11.45	6.88	3.00	19.50
LCOM	MVP	6	0.85	0.90	0.18	0.64	1.35
	MVVM	11	0.91	0.98	0.10	0.82	1.67
	MVI	9	0.97	1.02	0.07	0.66	1.50
WMC	MVP	6	5.00	6.00	5.00	3.00	11.00
	MVVM	12	12.50	17.58	11.88	1.00	84.00
	MVI	10	4.00	7.80	8.13	2.00	21.00

Higher CBO, LCOM-HS and WMC all indicate worse maintainability. LCOM is defined for fewer repositories because state holders with a single method have undefined cohesion.

MVI are statistically indistinguishable ($\delta = 0.10$, negligible), which is consistent with MVI being built upon the MVVM infrastructure on Android.

Cohesion (LCOM-HS). A consistent monotonic trend appears: cohesion degrades from MVP (0.85) through MVVM (0.91) to MVI (0.97, i.e., the least cohesive), with a large effect for MVP versus MVI ($\delta = -0.48$) and a medium effect for MVVM versus MVI ($\delta = -0.36$). The many small, interlinked state/intent/reducer classes of MVI appear to spread responsibilities at the cost of cohesion. LCOM is defined for fewer repositories (MVVM $n = 11$, MVI $n = 9$) because single-method state holders have undefined cohesion and are therefore excluded from this metric only.

Complexity (WMC). The picture inverts: MVVM concentrates logic in comparatively complex ViewModels (median WMC 12.5), whereas both MVP (5.0) and MVI (4.0) keep individual presentation classes simple (MVP vs. MVVM $\delta = -0.46$, medium). MVI thus trades higher coupling and lower cohesion for lower per-class complexity, distributing behaviour across more classes.

B. RQ2 – Change-Proneness

No statistically significant differences were found in either change frequency (commits per month) or churn per SLOC of the presentation layer (Kruskal–Wallis $p = 0.78$ and $p = 0.54$ respectively), and all pairwise effect sizes are negligible to small (Table V, Fig. 2). Within this corpus, the architectural pattern does not materially affect how frequently

TABLE IV
RQ1 – KRUSKAL-WALLIS AND PAIRWISE MANN-WHITNEY (HOLM)

Metric	Pair	p_{KW}	p_{Holm}	δ	Effect
CBO	MVP vs MVVM	0.069	0.080	-0.67	large
CBO	MVP vs MVI	0.069	0.164	-0.55	large
CBO	MVVM vs MVI	0.069	0.715	0.10	negligible
LCOM	MVP vs MVVM	0.179	0.421	-0.30	small
LCOM	MVP vs MVI	0.179	0.421	-0.48	large
LCOM	MVVM vs MVI	0.179	0.421	-0.36	medium
WMC	MVP vs MVVM	0.273	0.398	-0.46	medium
WMC	MVP vs MVI	0.273	0.783	0.10	negligible
WMC	MVVM vs MVI	0.273	0.493	0.30	small

* significant at $\alpha = 0.05$ after Holm correction.

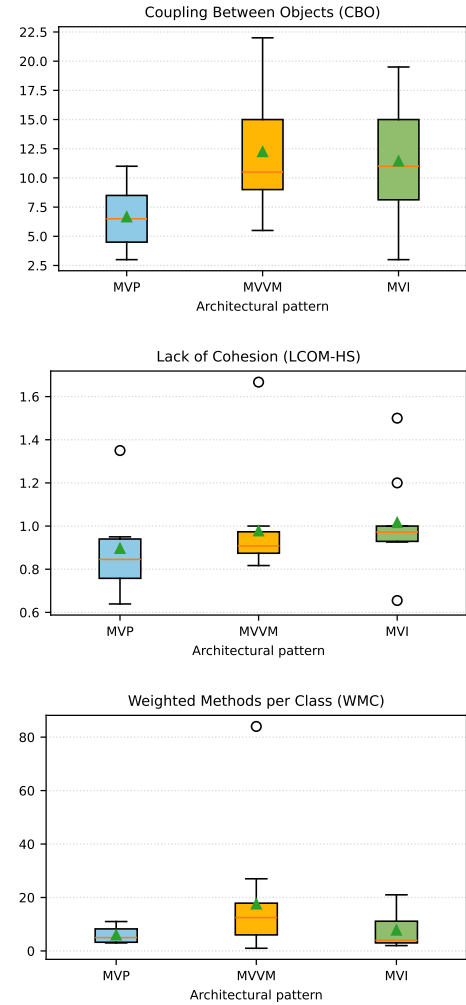


Fig. 1. Presentation-layer coupling (CBO, top), cohesion (LCOM-HS, middle) and complexity (WMC, bottom) by architectural pattern (per-repository medians). For LCOM-HS and WMC, higher is worse; the threshold from Filó et al. [4] is drawn as a reference where applicable.

the presentation layer is modified over the project history; evolution effort appears to be driven by factors other than the chosen pattern.

TABLE V
RQ2 – CHANGE-PRONENESS TESTS

Metric	Pair	p_{KW}	p_{Holm}	δ	Effect
Commits/month	MVP vs MVVM	0.776	1.000	-0.11	negligible
Commits/month	MVP vs MVI	0.776	1.000	-0.20	small
Commits/month	MVVM vs MVI	0.776	1.000	-0.12	negligible
Churn/SLOC	MVP vs MVVM	0.544	1.000	-0.08	negligible
Churn/SLOC	MVP vs MVI	0.544	1.000	0.17	small
Churn/SLOC	MVVM vs MVI	0.544	0.829	0.28	small

* significant at $\alpha = 0.05$ after Holm correction.

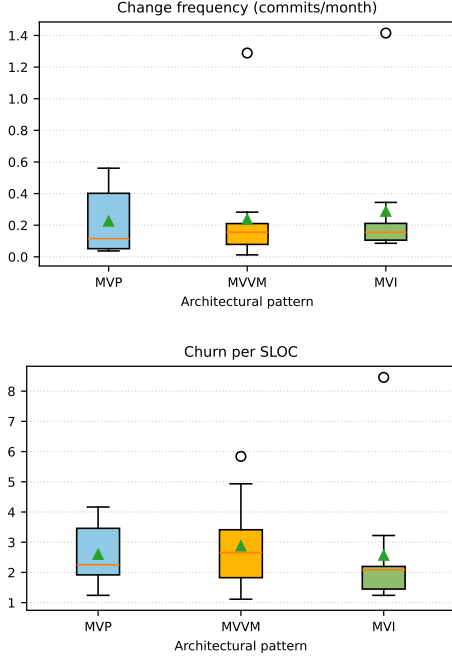


Fig. 2. Change-proneness of the presentation layer by architectural pattern (per-repository medians): change frequency in commits per month (top) and churn per SLOC (bottom). The distributions overlap heavily across patterns.

VII. DISCUSSION

Taken together, the two research questions paint a coherent and somewhat counter-intuitive picture of how the choice of architectural pattern relates to the maintainability of the Android presentation layer.

On RQ1, structure is shaped less by the pattern label than by where each pattern places its logic. The clearest structural separation is between MVP and the two modern patterns: MVP exhibits substantially lower *median* coupling, with large effect sizes, even though the omnibus test stays just above significance because the groups are small. MVVM and MVI, in contrast, are statistically indistinguishable in coupling. This is consistent with the observation, confirmed during corpus construction, that on Android MVI is typically *implemented on top of* MVVM (a *ViewModel* that exposes an immutable *StateFlow*); the two are points on a continuum rather than competing families. Their real difference is one of *distribution* of behaviour: MVVM concentrates logic in fewer but more complex *ViewModels* (high WMC), whereas MVI spreads it

across many small state, intent and reducer classes that lower per-class complexity at the cost of higher coupling and lower cohesion. In other words, MVI does not remove complexity, it relocates it from inside a class to the relationships between classes.

On RQ2, the pattern does not explain how the presentation layer evolves. Neither change frequency nor churn differs across patterns. Within this corpus, how often presentation code is rewritten appears to be governed by product and team factors (domain, feature pressure, contributor turnover) rather than by the architectural style, echoing the warning of Kalliamvakou et al. [3] that observational repository data captures many confounds at once.

These results refine, rather than contradict, prior work. The qualitative comparison of Vijaywargi and Boddapati [1] credits MVI with greater state predictability; our quantitative view adds a caveat, namely that this predictability comes with measurably lower cohesion and higher coupling in the state-holding classes, so “cleaner” is not the same as “more cohesive.” Pradana et al. [2] found runtime memory differences between MVVM and MVP on a single controlled project; our cross-project structural evidence is orthogonal to that finding and shows that, structurally, MVVM and MVI are far closer to each other than either is to MVP.

Implications for practitioners. For teams already on the Architecture Components stack, the decision between MVVM and MVI is unlikely to be settled by raw structural metrics, since the two are nearly equivalent in coupling; it should instead be driven by considerations such as state-management predictability, testing strategy and team familiarity. The single most actionable signal we observe is that *ViewModel* complexity (WMC) can grow large under MVVM, so teams that deliberately keep *ViewModels* small recover most of the structural benefit that MVI obtains by construction, without fragmenting the codebase.

VIII. THREATS TO VALIDITY

Construct validity. CBO is computed as a syntactic coupling proxy (distinct referenced types) rather than through full type resolution; the strong correlation with CK ($\rho = 0.83$) mitigates this concern. LCOM has several incompatible definitions in the literature; we fix and report the Henderson-Sellers variant.

Internal validity. Pattern membership is inferred from code signals and may misclassify hybrid designs; the MVI/MVVM overlap is intrinsic to Android and is handled by an explicit precedence rule and a manual audit. Aggregating to per-repository medians prevents large repositories from dominating the analysis. Because the available MVP applications are systematically smaller and less popular than the MVVM/MVI ones, MVP’s lower coupling and complexity may be partly a *size* effect rather than a pure pattern effect; the MVP results should therefore be read as indicative rather than conclusive.

External validity. The corpus is biased toward popular, well-maintained open-source projects, which are not representative of all Android software [3]. Genuine MVP applications

are scarce and small, yielding an unbalanced and limited MVP sample; results for that pattern must be read with caution.

Conclusion validity. Group sizes are modest, especially for MVP; we therefore report effect sizes (Cliff’s δ) alongside p -values and apply multiple-comparison correction.

IX. CONCLUSIONS AND FUTURE WORK

This work presented a reproducible empirical study comparing the impact of the MVP, MVVM and MVI architectural patterns on the maintainability of the presentation layer of real Android applications, combining repository mining, uniform Kotlin/Java static analysis and version-history change-proneness over a code-confirmed corpus of 28 applications.

The study answers its two research questions directly. **RQ1:** MVP shows the lowest *median* presentation-layer coupling (large effect sizes, though the omnibus test is borderline), but MVVM and MVI are structurally indistinguishable in coupling and differ only in how they distribute logic, MVVM in fewer complex ViewModels and MVI across many simpler but less cohesive classes. **RQ2:** the architectural pattern does not explain the change-proneness of the presentation layer; no significant differences arise in either change frequency or churn. A notable secondary finding is that MVP has effectively been abandoned in popular open-source Android development: nearly every highly starred “MVP” repository is in fact an architecture library, and genuine MVP applications are few and small, which also makes the MVP results indicative rather than conclusive.

As future work, the runtime-performance dimension (CPU, memory and response time) that cannot be mined from repositories could be measured by building and profiling benchmark applications on an emulator, and the analysis could be extended to multiplatform frameworks (Kotlin Multiplatform, Flutter) and to defect density mined from issue trackers.

DATA AVAILABILITY

The complete material of this study, including the mining and analysis pipeline (Python and Java scripts), the curated corpus, the per-class and per-repository metrics, the statistical outputs and all figures, is openly available so that the results can be reproduced: <https://drive.google.com/REEMPLAZAR-CON-ENLACE>.

ACKNOWLEDGMENT

The author thanks the teaching staff of the Mobile Applications course at the Universidad Nacional de Costa Rica for their guidance throughout this work.

USE OF ARTIFICIAL INTELLIGENCE

An AI coding agent was used to support the empirical part of this study: it researched and designed how to perform the repository mining, carried out the mining, wrote and executed the validation tests, and returned the resulting measurements, all computed from the real source code and version history of the analyzed repositories. The author was responsible for writing this paper and for processing and interpreting the

results produced by the experiment. AI assistance was also used for language and formatting (IEEE style and grammar). No data was generated or fabricated by AI: every metric reported here was mined directly from the real repositories.

REFERENCES

- [1] A. Vijaywargi and U. K. Boddapati, *Architectural Patterns in Android Development: Comparing MVP, MVVM, and MVI*, International Journal for Research in Applied Science and Engineering Technology (IJRASET), vol. 12, no. 4, pp. 4611–4616, Apr. 2024.
- [2] F. Pradana, R. I. Langundi, D. Pramono, and N. I. Iriani, *Comparative Analysis of MVVM and MVP Patterns Performance on Android Dashboard System*, Jurnal Nasional Teknik Elektro dan Teknologi Informasi (JNTETI), vol. 14, no. 2, pp. 87–95, May 2025.
- [3] E. Kalliamvakou, G. Gousios, K. Blincoe, L. Singer, D. M. German, and D. Damian, *The Promises and Perils of Mining GitHub*, in *Proc. 11th Working Conf. on Mining Software Repositories (MSR 2014)*, Hyderabad, India, 2014, pp. 92–101.
- [4] T. G. S. Filó, M. A. S. Bigonha, and K. A. M. Ferreira, *Evaluating Thresholds for Object-Oriented Software Metrics*, Journal of the Brazilian Computer Society, vol. 30, no. 1, Sep. 2024.
- [5] S. R. Chidamber and C. F. Kemerer, *A Metrics Suite for Object Oriented Design*, IEEE Transactions on Software Engineering, vol. 20, no. 6, pp. 476–493, Jun. 1994.
- [6] B. Henderson-Sellers, *Object-Oriented Metrics: Measures of Complexity*. Upper Saddle River, NJ, USA: Prentice-Hall, 1996.
- [7] M. Aniche, *Java code metrics calculator (CK)*, 2015. [Online]. Available: <https://github.com/mauricioaniche/ck>
- [8] F. Khomh, M. Di Penta, and Y.-G. Guéhéneuc, *An Exploratory Study of the Impact of Code Smells on Software Change-proneness*, in *Proc. 16th Working Conf. on Reverse Engineering (WCRE)*, 2009, pp. 75–84.
- [9] C. Wohlin, P. Runeson, M. Höst, M. C. Ohlsson, B. Regnell, and A. Wesslén, *Experimentation in Software Engineering*. Berlin, Germany: Springer, 2012.